Final PRD: Developer Productivity Dashboard

> **Document Owner:** Navneet Shukla | **Last Updated:** 5-Jun-2026

Metadata Field	Value
Jira Epic	Link epic here
Delivery Date	TBD — to be set after PRD sign-off
Document Status	Draft
Product Lead	Navneet Shukla
Engineering Lead	TBD
Design Lead	TBD
QA Lead	TBD
Relevant Documents	Developer-Dashboard CLI Repo

Table of Contents

1. Overview

Background

Company_Name's engineering team has adopted Claude Code as its daily developer tool. To measure the return on this AI investment, engineering managers need to correlate developer-specific AI spend and code generation with actual JIRA deliverables. Currently, a Python CLI tool (`merge.py`) processes monthly exports into a flat CSV. However, this is terminal-bound, has no history/trends, is manual to run, and is inaccessible to non-technical stakeholders or developers who want to track their own progress.

Why are we doing this?

We are making a company-level bet on AI-assisted development. Without a web-based dashboard, we cannot easily:
* Identify who is actively adopting AI vs. coding manually.
* Track whether AI usage translates to more deliverables, faster.
* Monitor cost-per-deliverable and see trends over time.
* Empower developers to see their own metrics and drive self-improvement.

How do we know this is a real problem worth solving?

- The backend metrics formulas and bug-attribution logic are already built and validated via a CLI proof-of-concept.
- Sharing flat CSVs is cumbersome. Managers have to run CLI scripts and share static files over Slack.
- Developers currently have zero visibility into their own numbers.

- No historical trend data is stored (the CLI overwrites its output on each run).

How does this fit into company objectives?

- **ROI Measurement**: Directly validates the company's spend on Claude Code seats.
- **Engineering Quality**: Attributes bugs to stories to identify quality risks early.
- **Transparency**: Fosters a developer self-improvement loop.

Who are we solving for?

- **Engineering Managers**: Need team-wide leaderboards, productivity trends, and flag alerts to spot underperformers or quality risks.
- **Developers**: Need a personal page showing their token spend, efficiency score, and trends over time.

---

2. Goals

Build a React-based web dashboard that aggregates Claude Console usage and JIRA deliverables per developer—supporting both weekly and monthly periods—with persistent historical data and multiple file ingestion paths.

What success looks like:

1. Managers and developers can view the current week/month leaderboard and trend statistics in a browser.
2. Users can drill down into individual developer profiles to see trend charts.
3. Data can be refreshed either automatically from server files or by dragging and dropping exports in the UI.
4. Historical data accumulates so trends remain visible across multiple periods.

---

3. Success Metrics

- **Dashboard Adoption**: $\geq 80\%$ of the engineering team views the dashboard at least once per month within 60 days of launch.
- **Manual CSV Sharing Eliminated**: Zero manual output.csv files shared via Slack/email after launch.
- **Data Freshness**: Under 60 seconds from pipeline execution to dashboard updates.
- **Developer Self-Service**: $\geq 60\%$ of developers check their profile pages without prompting.

---

4. Assumptions & Dependencies

Assumptions

- Developers use matching emails in Claude Console and JIRA (bridged by `developers.csv` mapping if mismatched).
- JIRA exports contain necessary fields: Assignee email, Issue type, Story points, Resolved/Created date, Sprint, and Epic.
- The existing Python pipeline logic (`src/metrics.py`, `src/guardrail.py`, etc.) is correct and will be imported directly by the FastAPI backend.
- Authentication and role-based access are out of scope for v1. All users can view all data on the internal network.

Dependencies

- **FastAPI Backend**: To host REST APIs, handle file uploads, and execute the Python metrics engine.
- **SQLite Database**: To store raw transactions, historical run histories, configuration settings, and processed metrics.
- **React SPA (Vite)**: To render the responsive frontend, charts, and upload log.
- **Chart.js or Recharts**: To render historical line and stacked bar charts.

5. Requirements

5.1 Ingestion & Administration

#	Title	User Story	Acceptance Criteria	Priority	Notes
1	**Web UI CSV Upload**	As an admin, I want to upload a Claude CSV and JIRA CSVs together via the UI.	Form accepts Claude usage CSV, JIRA deliverables CSV, and JIRA bugs CSV. User selects period (week or month) and submits to execute parsing.	**Must Have**	Validates col headers. Rej files are miss
2	**Server Directory Ingestion**	As an admin, I want to trigger runs from files placed directly on the server.	A “Run from Server Files” button runs the pipeline on files placed in a configured server directory. Also runs via a monthly cron job.	**Must Have**	Defaults to scanning a p defined folde (e.g. `./data_`
3	**Upload & Run Log**	As an admin, I want to see a log of past pipeline runs.	A log table showing timestamp, period type, trigger type (manual web upload / server files / cron), status, and error logs if failed.	**Must Have**	Retains last 2 in database.
4	**Jira Base URL Config**	As an admin, I want to configure the Jira link prefix so that issue keys link to my Jira site.	Admin settings panel in UI allows saving `jira_base_url` (e.g. `https://Company_Name.atlassian.net/browse/`) to the database.	**Must Have**	If blank, issu in UI render a plain text.

5.2 Team Dashboard (Leaderboard)

#	Title	User Story	Acceptance Criteria	Priority	Notes
5	**Leaderboard Table**	As a manager, I want to see a ranked list of all developers.	Table displays: Developer Name, Claude Spend, Lines of Code, Deliverables, Story Points, AI Efficiency, Bugs, Bug Rate, and Flag.	**Must Have**	Default sort: Flag severity (Red → Gray → Yellow → Green), then name.
6	**Granular Period Selector**	As a manager, I want to switch between weekly and monthly data.	Toggles for “Weekly” or “Monthly” view. Dropdown lists all stored periods (e.g. `2026-W23` or `2026-05`). Selecting reloads dashboard.	**Must Have**	Weekly format: `YYYY-Www`. Monthly: `YYYY-MM`.
7	**Flag Filter**	As a manager, I want to filter the table by flags.	Multi-select checkbox above table allows filtering developers (e.g., show only Red + Gray developers).	**Must Have**	Default: Show all.
8	**Team Summary Cards**	As a manager, I want to see aggregate team numbers.	4 cards displaying: Total Tokens/Spend, Total Deliverables closed, Team Average Efficiency, and Flag Distribution counts.	**Must Have**	Update dynamically with period selection.

5.3 Developer Profile & Trends

#	Title	User Story	Acceptance Criteria	Priority	Notes
9	**Developer Profile Page**	As a developer, I want a dedicated space to	Clicking any name in the leaderboard opens their profile,	**Must Have**	Explanation explains *why* they

#	Title	User Story	Acceptance Criteria	Priority	Notes
		view my performance.	showing cards with current period metrics and a plain-text flag explanation.		got their flag.
10	**Historical Trend Chart**	As a developer, I want to see my progress over time.	Interactive line chart displaying trend lines for `ai_efficiency_score`, `spend_usd`, and `deliverables_count` across stored periods.	**Must Have**	Toggles between last 8 weeks or last 6 months.
11	**Bug Attribution Detail List**	As a developer, I want to see which bugs were attributed to me.	Section lists each attributed bug key, epic, sprint, creation date, and the parent story that triggered the 30-day link.	**Must Have**	Bug keys link to JIRA using configured Jira Base URL.
12	**Team Aggregate Trends Page**	As a manager, I want to see team adoption and quality trends over time.	Page displaying team-level aggregate line charts for total spend, efficiency, deliverables, and bug counts across all stored history.	**Must Have**	Includes an “AI Adoption Rate” card.

6. UX Mockups

The system requires four core views in the React application:
1. **Leaderboard View**: Team summary cards at the top, period/weekly/monthly selector toggles, flag filter checkboxes, and the sortable developer leaderboard table.
2. **Developer Profile View**: Selected developer stats, plain-language flag diagnostic card, and the line charts showing trends.
3. **Data Management View**: File uploader (drag-and-drop form), “Run Server Ingestion” trigger button, settings inputs (Jira Base URL, developers mapping editor), and the pipeline run log table.
4. **Team Trends View**: Multi-line trends of team KPIs and a stacked bar chart of flag distributions over time.

7. Questions

- **Weekly Claude Spend Fallback**: Since raw Claude CSVs only export monthly totals, how do we handle weekly views?
  - *Outcome*: The backend will allow uploading weekly exports if available. If a monthly Claude CSV is uploaded, the backend will evenly distribute the monthly spend across the weeks of that month (divide by 4.3) for the weekly leaderboard views and display an info indicator stating “Claude spend approximated from monthly data”.

8. Out of Scope

- Authentication and login for the MVP.
- Role-based access control (RBAC).
- Direct API integrations with Anthropic Usage API and JIRA REST API (deferred to Phase 2).
- Mobile layouts (Desktop-first internal dashboard).
- Alerts via Email or Slack.